EXPERIMENT NO: 6

Student Name: Geetika
UID: 23BCS12885
Branch: BE-CSE
Section/Group: KRG-3A
Semester: 6th
Date of Performance: 26/02/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim

To design a highly scalable video streaming platform similar to YouTube that supports adaptive bitrate streaming (HLS/DASH), dynamic manifest handling, live streaming with sliding window playback, and real-time engagement features (likes and comments) with high availability and low latency.

Objectives

- Understand Adaptive Bitrate Streaming using HLS and DASH protocols.
- Analyze the role of manifest files in dynamic bandwidth handling.
- Design architecture for both Video-on-Demand (VOD) and Live Streaming.
- Implement scalable like and comment systems for static and live videos.
- Apply event-driven architecture for high-volume engagement processing.
- Ensure fault tolerance, high availability, and eventual consistency.

Procedure

1. Studied real-world streaming protocols including HLS and DASH.
2. Analyzed manifest file structure (.m3u8 and .mpd) and adaptive bitrate switching.
3. Designed live streaming architecture using sliding window manifest approach.
4. Implemented scalable like system using Redis counters and asynchronous persistence.
5. Designed comment service using NoSQL partitioning for large-scale videos.
6. Integrated WebSocket-based real-time chat system for live streaming.
7. Evaluated CDN-based global content distribution for low-latency playback.

Functional Requirements

1. Users should be able to register and login to the platform.
2. Content creators should be able to upload videos.
3. The system should process uploaded videos into multiple resolutions.
4. The system should generate HLS/DASH manifests for adaptive streaming.
5. Users should be able to watch videos.
6. Users should be able to search videos by title or metadata.
7. Users should be able to like or dislike videos.

Core Entities of the System

• Users
• Videos
• VideoSegments
• ManifestFiles
• LiveStreams
• Interactions (Likes/Dislikes)
• Comments
• ChatMessages

API Design

1. Authentication API

- **Register**: POST /api/v1/auth/register
- **Login**: POST /api/v1/auth/login

2. Video & Feed API

- **Initialize Upload**: POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
- **Get Video Metadata**: GET /api/v1/videos/{video_id}
- **Get Personalized Feed**: GET /api/v1/feed (Pagination Supported)
- **Search Videos**: GET /api/v1/search?q={query}

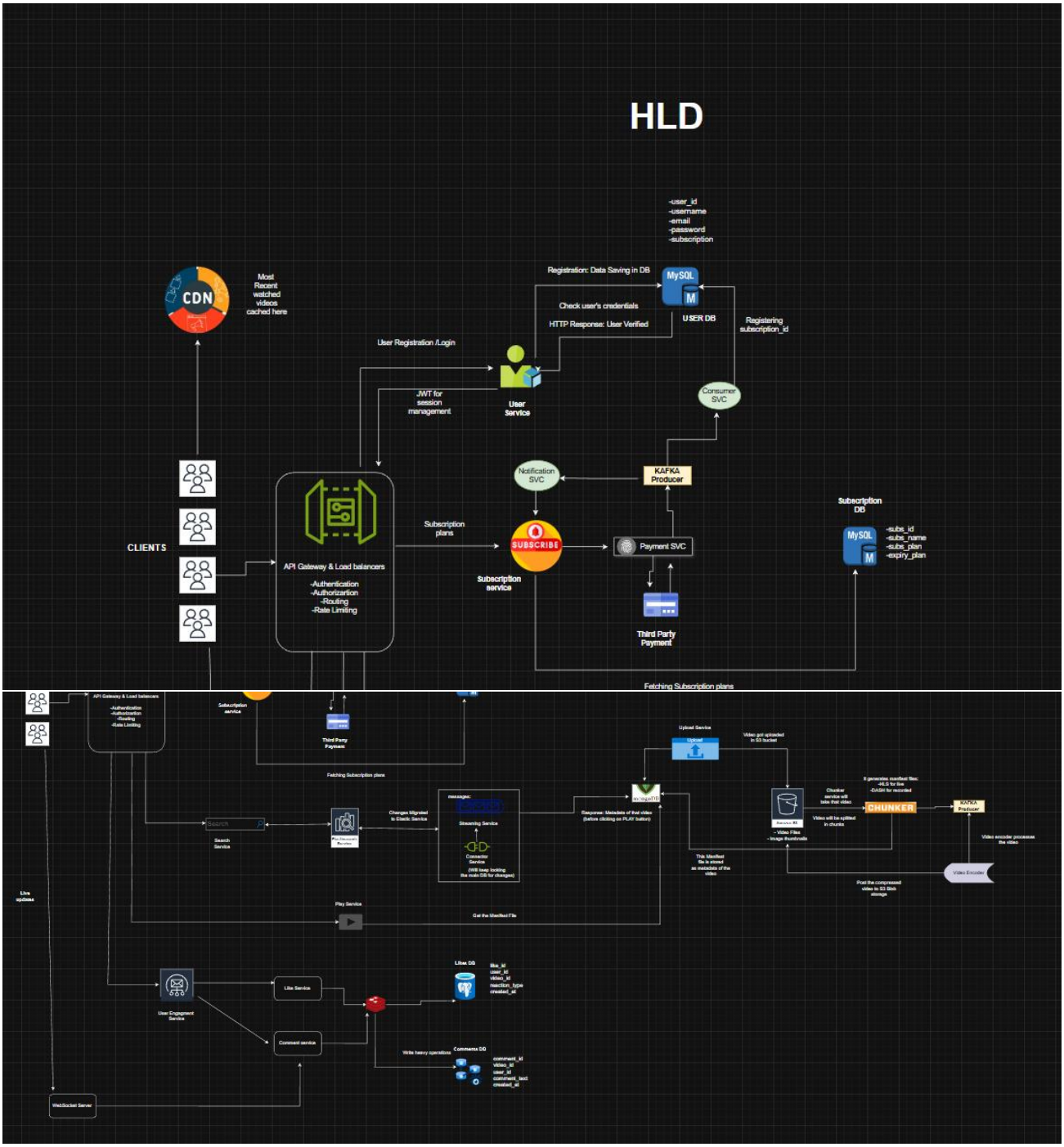
3. Interaction API

- **Like Video**: POST /api/v1/videos/{video_id}/like
- **Add Comment**: POST /api/v1/videos/{video_id}/comments
- **Get Comments**: GET /api/v1/videos/{video_id}/comments

Non-Functional Requirements

1. System must handle extremely high throughput for concurrent uploads and millions of simultaneous views.
2. High Availability must be prioritized, ensuring no single point of failure.
3. System should follow an Eventual Consistency model for social metrics (view counts, likes) to optimize database write loads.
4. Video delivery latency should be minimized via Edge Caching and a Global CDN.
5. Transcoding pipeline must be horizontally scalable to handle viral upload spikes.

High Level Design (HLD)



Outcome

- Designed a scalable YouTube-like architecture supporting HLS and DASH.
- Implemented dynamic manifest refresh for live streaming.
- Designed sliding window mechanism for low-latency live playback.
- Built scalable like and comment systems for static and live videos.
- Applied event-driven architecture for engagement processing.
- Understood CDN-based global content delivery and adaptive bitrate streaming.